CSE 102 Assignment 5 Report

Objective:

The objective of this project is to implement a Mastermind-style number guessing game using C programming language. The game features both an Admin Mode for configuration setup and a Player Mode for playing the game with feedback and scoring.

Features Implemented:

- Dynamic memory allocation via malloc() and free() for both secret code and guesses
- Reading configuration from vault_config.txt
- Generating or reading a secret code from vault_code.txt
- Logging all game actions and scores to game_log.txt
- Giving feedback with 'C', 'M', 'W' indicators for Correct, Misplaced, and Wrong digits
- Calculating scores per attempt and total score with rank titles

Core Functions:

int* generate_code()

Generates or reads the secret code dynamically using config settings:

```
int* generate_code() {
    struct Config config = read_config();
    ...
    code = malloc(config.code_length * sizeof(int));
    ...
}
```

This function ensures dynamic memory allocation and supports both duplicate and non-duplicate digit options depending on user input.

int* get_guess()

Prompts the user to enter a guess and returns it as a dynamically allocated array:

```
int* get_guess() {
    struct Config config = read_config();
    ...
    guess = malloc(config.code_length * sizeof(int));
    ...
}
```

Input validation includes digit count and digit range check.

char* calculate_feedback(...)

Evaluates the user's guess and returns a feedback string:

```
feedback[i] = (guess[i] == code[i]) ? 'C' : ' ';
...
if (guess[i] == code[j]) feedback[i] = 'M';
...
feedback[i] = 'W';
```

This function ensures proper feedback order aligned with guess digits.

Outputs:

```
Enter mode (A for Admin, P for Player): p
[DEBUG] Secret Code: 5382

Attempt 1/10
Enter your guess: 5283
Guess:     5    2    8    3
Feedback: C    M    C    M    | Score: 30

Attempt 2/10
Enter your guess: 1111
Guess:     1    1    1    1
Feedback: W    W    W    W    | Score: 22

Attempt 3/10
Enter your guess: 5832
Guess:     5    8    3    2
Feedback: C    M    M    C    | Score: 52

Attempt 4/10
Enter your guess: 5382
Guess:     5    3    8    2
Feedback: C    C    C    C    | Score: 92

Final Score: 92
Rank: Code Master 🧠


...Program finished with exit code 0
Press ENTER to exit console.
```

```
--- Vault Codebreaker Game Log ---
Game Date: Fri Apr 18 20:22:13 2025
Secret Code: 5382
Code Length: 4
Digit Range: 0-9
Duplicates Allowed: 1
Max Attempts: 10

Attempt 1: 5283 => Feedback: C M C M | Attempt Score: 30 | Total Score: 30
Attempt 2: 1111 => Feedback: W W W W | Attempt Score: -8 | Total Score: 22
Attempt 3: 5832 => Feedback: C M M C | Attempt Score: 30 | Total Score: 52
Attempt 4: 5382 => Feedback: C C C C | Attempt Score: 40 | Total Score: 92

Final Score: 92
Rank: Code Master 🧠
```

main.c | vault_code.txt | vault_config.txt | game_log.txt

```
1  5382
```

main.c | vault_code.txt | vault_config.txt | game_log.txt

```
CODE_LENGTH=4
DIGIT_MIN=0
DIGIT_MAX=9
ALLOW_DUPLICATES=1
MAX_ATTEMPTS=10
POINTS_CORRECT=10
POINTS_MISPLACED=5
PENALTY_WRONG=2

```

Rank System:

Based on total score:

- 90+: Code Master
- 70+: Cipher Hunter
- 50+: Number Sleuth
- ...

Implemented in the end of player_mode() using:

```
char *rank = total_score >= 90 ? "Code Master " : ...;
```

Conclusion:

This project has improved understanding of:

- File I/O in C
- Dynamic memory usage
- Struct-based configuration
- Feedback systems for game design

The code meets all project requirements including function signatures, dynamic allocation, file handling, and structured feedback logic.